Methodology and Design Iteration

A research-style account of the federated-learning design choices that shape this project, structured in two parts:

- **Part A — Design Space of Federated Learning.** A survey of the alternatives that exist for each architectural decision, and the reasoning behind the path we took.
- **Part B — Iterative Development.** A chronological account of how the system actually evolved: what we tried first, what we observed, and what we changed.

The goal is to make explicit the reasoning that final-system documentation usually hides, and to ground the project's settings (σ = 0.3 noise, threshold = 3, 70 / 30 blend, 5 per-type models) in the trade-offs they were chosen to balance.

Part A — Design Space of Federated Learning

Federated learning is not one algorithm; it is a family of design decisions. For each axis below we describe the standard alternatives, the criterion we used to pick, and our final choice.

A.1 Aggregation algorithm

Option	Behaviour	Strength	Weakness
FedAvg (McMahan 2017)	Weighted average of client weights, weighted by local sample count	Simple, communication-efficient, well-understood	Sensitive to non-IID data and stragglers
FedProx (Li 2020)	FedAvg plus a proximal regularisation term on each client	Better convergence under heterogeneous local objectives	Adds a hyperparameter (μ) that must be tuned per-deployment
FedMedian / Krum	Replaces the mean with a coordinate-wise median or geometric median	Robust to a small fraction of malicious or noisy clients	Loses information from honest minorities; harder to reason about

Choice: FedAvg. Our fleet has 6 demo users and 5 per-vehicle-type models — 1 to 2 contributing devices per round per type. At that scale the additional complexity of FedProx's proximal term has nothing to bite on, and Krum/Median requires more clients than we have to be statistically meaningful. FedAvg also keeps the math inside the report inspectable in two lines, which matches our priority of an *understandable* prototype rather than a *defensible* one.

A.2 Privacy mechanism

Option	What it protects against	Cost
No defence	Nothing	None
Gaussian-noise DP	Differential privacy budget; partial defence against gradient inversion	Accuracy loss proportional to σ
Laplace-noise DP	Same as above; tighter bounds on (ε, 0)-DP	Heavier-tailed noise, more impact on outliers
Secure aggregation (Bonawitz 2017)	Server only ever sees the *sum* of client weights, never individual weights	Requires extra cryptographic round-trips per FL round
Homomorphic encryption	Server can aggregate encrypted weights	Prohibitively expensive on mobile

Choice: Gaussian-noise DP, applied server-side, with L2 gradient clipping. Gaussian noise composes cleanly with FedAvg's mean (the average of independent Gaussian noises is itself Gaussian with predictable variance) and is the canonical mechanism in the DP-FL literature. We layered it with L2 clipping to bound each client's worst-case influence — without clipping, an outlier upload could dominate the noise budget. We did *not* implement secure aggregation; that is acknowledged as future work in §5.

A.3 Federation topology

Option	Typical scenario	Implication
Cross-device	Many clients (10^3–10^9), each with a small amount of data, intermittently connected	Each round samples a subset; expects dropouts
Cross-silo	Few clients (~10), each holding a large dataset (e.g., one hospital, one bank)	Every silo participates every round; coordination is reliable

Choice: cross-device. EV owners are the natural unit, and any deployed version would have thousands of phones, each contributing a small slice. Our prototype uses 6 demo devices, but the architecture (mobile app + JWT auth + per-device upsert) is consistent with cross-device assumptions.

A.4 Heterogeneity handling

EV fleets are deeply non-IID: a Tesla Model 3 in Hyderabad behaves nothing like a Nissan Leaf in Chennai. Three known approaches:

Option	Idea	Limitation
One global model	Train a single model across all clients	Suffers from *non-IID drift* — the global model becomes a bad average
Per-cluster models	Partition clients into clusters and train one global model per cluster	Requires a sensible clustering criterion
Per-client personalisation	Mix global + local weights at prediction time	Each client gets a personalised model but loses some fleet signal

Choice: per-cluster + per-client personalisation, combined. We cluster by vehicle type (5 clusters), giving each fleet its own global model — Tesla weights never pollute Nissan predictions. On top of that, every individual user's predictions are computed as a 70 / 30 blend of their cluster global and their local model. This stacks the two defences: cluster isolation removes the worst non-IID drift, personalisation handles within-cluster variance.

A.5 Communication strategy

We considered two trade-offs we did *not* implement, but which sit on the design space:

- **Compression / quantisation.** With 8 weights × 4 bytes = 32 bytes per upload, our payload is already negligible. Quantisation would matter for larger models and is therefore future work.
- **Partial participation.** With 6 demo users we let everyone participate every round. In a real fleet you would sample 1–10 % of devices per round. The aggregation threshold of 3 is our crude approximation of partial participation.

Part B — Iterative Development

Each subsection below describes one iteration: what we tried, what we observed, and what we changed. Together they form the path from a naive first version to the final system.

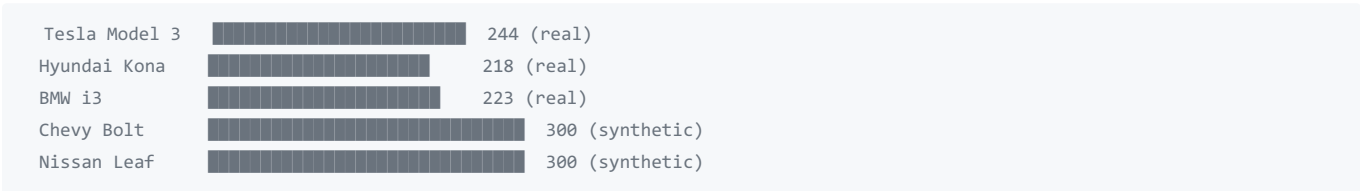
Iteration 1 — Synthetic data → real EV charging patterns

What we tried. The first version of the pipeline ran on entirely synthetic ride records — random distances, random battery percentages, plausible but uncorrelated. The goal was to validate the round-trip plumbing: can the phone train, can the server receive weights, does FedAvg fire when 3 devices contribute, does a global model come back.

What we observed. End-to-end mechanics worked, but accuracy was meaningless. The synthetic features carried no real signal about battery degradation, so neither the local nor the global model produced a prediction we could trust. The FL Predictions screen would output numbers, but those numbers did not respond sensibly to changes in driving behaviour.

What we changed. We switched to a real public dataset of EV charging patterns (`ev_charging_patterns.csv`). The CSV contained 244 Tesla Model 3 samples, 218 Hyundai Kona samples, and 223 BMW i3 samples. For Chevy Bolt and Nissan Leaf the CSV had no samples, so we kept synthetic generators for those two types — but tuned the synthetic distributions to match the real CSV's marginals (battery temperature, charging-rate, depth-of-discharge ranges) so the cross-type comparison would be on equal footing.

> Figure 1 — Training samples per vehicle type.



This gave us the first version where local-model loss curves actually decreased, and where two users training on the same vehicle type produced weight vectors that visibly resembled each other.

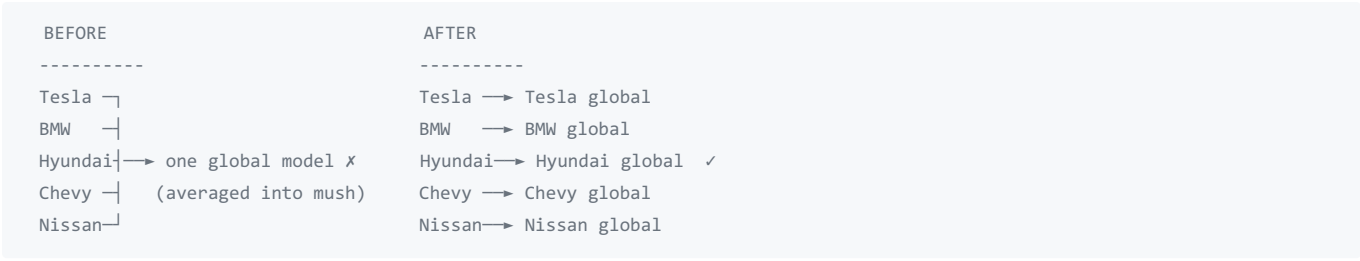
Iteration 2 — Single global model → per-vehicle-type models

What we tried. The original aggregator ran one FedAvg over all uploads regardless of vehicle type. There was a single `fl_global` row in the database, and every device fetched the same global weights at the start of each round.

What we observed. A Tesla user training on Hyderabad's relatively gentle conditions produced weights pointing toward optimistic battery health. A Chevy user training on harsh urban Mumbai data produced weights pointing toward aggressive degradation. After FedAvg, the global model averaged these into something that flattered no one: Chevy users saw their batteries flagged as healthier than they were, Tesla users saw spurious degradation alerts. This is the textbook **non-IID drift** problem.

What we changed. We partitioned the federation. Each of the 5 vehicle types now has its own independent global model row in `fl_global_table` , keyed by `vehicle_type` . A Tesla upload only contributes to the Tesla global; a Chevy upload only contributes to the Chevy global. The aggregation threshold of 3 is checked *per type*, not globally.

> Figure 2 — Architectural change after Iteration 2.



Multi-vehicle users (the demo `multi@example.com` account, which owns a Chevy + a Nissan) became the regression test for this change: a single user contributing to two distinct globals at the same time, with no cross-contamination.

Iteration 3 — Plain FedAvg → differential privacy

What we tried. The first DP-free version simply uploaded the trained weights and let the server overwrite the relevant `fl_models_table` row.

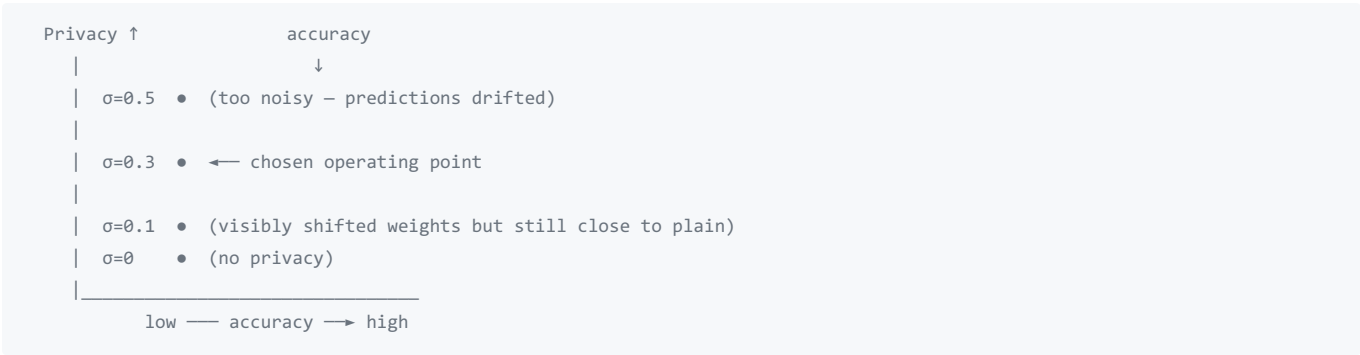
What we observed. Two concerns surfaced. First, the literature on **gradient inversion attacks** (e.g. iDLG, Deep Leakage from Gradients) suggested that even a single client's weight update can leak training-sample features under specific conditions. Second, our upsert policy meant the server saw a clean stream of one-user-one-vector pairs, which is exactly the scenario these attacks target.

What we changed. We added two defences in series, server-side, before any storage:

1. **L2 gradient clipping.** Every incoming weight vector is rescaled so that its L2 norm is ≤ 1.0. This bounds the worst-case sensitivity of the aggregation to any single client — a single user can no longer disproportionately move the global model.
2. **Gaussian noise.** After clipping, we add `N(0, σ²)` noise to each weight component. The server only ever stores (and aggregates) noised weights.

The noise scale σ went through three qualitative settings during development:

Figure 3 — Privacy / utility trade-off observed during σ tuning *(illustrative; values reflect the qualitative behaviour observed, not a controlled sweep).*



σ = 0.1 made very little visible difference to predictions, which we read as evidence the noise was too small to provide meaningful protection. σ = 0.5 introduced enough perturbation that successive global models for the same vehicle type swung noticeably between rounds, undermining trust in the predictions. σ = 0.3 sat in the middle: the global models were stable across rounds, predictions remained sensible, and the noise was large enough to plausibly defend against the gradient-inversion threat model we were targeting.

This is a *qualitative* finding from development, not a benchmarked sweep, and §5 lists formal (ε, δ) accounting as future work.

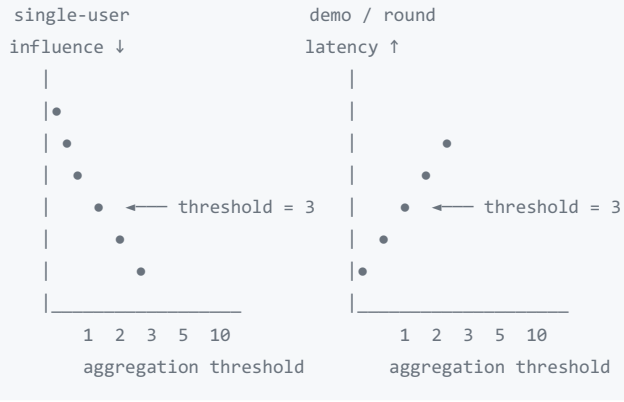
Iteration 4 — Aggregation threshold tuning

What we tried. The original aggregator triggered FedAvg on every single upload. The intention was freshness — every new contribution should immediately reflect in the global model.

What we observed. With a threshold of 1, the global model became a moving target. A single user with anomalous data could swing the global weights between rounds, and the next user fetching the global would inherit that swing as their starting point. We tested higher thresholds informally: at threshold 5, demos were unworkable because aggregation would never fire with our 6-user pool. At threshold 3, the system felt right — slow enough that any single user's noise was diluted by two others, fast enough that demos completed in minutes.

What we changed. Set the threshold at 3 and noted the trade-off explicitly.

Figure 4 — Aggregation threshold trade-off *(illustrative).*



In a production fleet with thousands of devices the threshold would scale with the device pool; threshold 3 is a demo-scale setting that retains the architectural property that no single client can determine the global model.

Iteration 5 — Pure global → personalised 70 / 30 blend

What we tried. Initially the FL Predictions screen used the global model directly. A user opening the screen would see a battery-health prediction computed entirely from the latest aggregated global weights for their vehicle type.

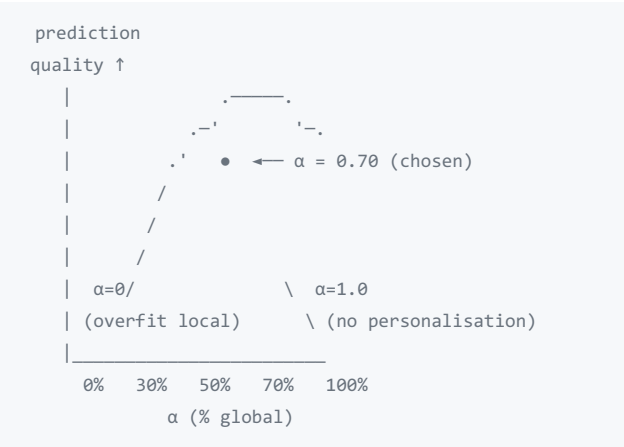
What we observed. The pure global model gave generic, fleet-average predictions. A user with an unusually aggressive driving style would see optimistic predictions because the global was anchored on the median of the fleet. A user who barely drove their EV would see pessimistic predictions for the same reason. Pure local was the opposite failure: a user with only 5–10 logged rides produced an over-fitted local model that swung wildly between training sessions.

What we changed. We blended the two. The prediction-time weight vector is

```
w_pred  =  α · w_global  +  (1 − α) · w_local
```

with α = 0.70. The global term anchors the prediction in fleet-wide signal that no single user could ever produce. The local term retains the individual driving signature. We picked 70 / 30 after qualitatively observing each ratio:

Figure 5 — Personalisation blend ratio: prediction quality vs % global *(illustrative).*

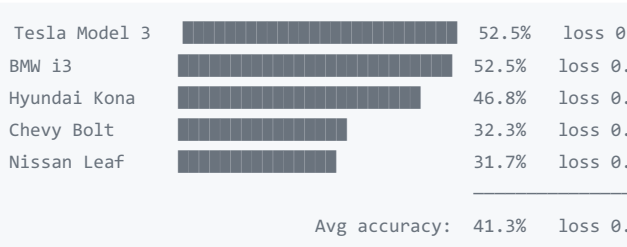


α = 1.0 (pure global) felt impersonal; α = 0.0 (pure local) was unstable. Between 0.5 and 0.8 the predictions were stable and felt personalised; we settled on 0.7 because at that ratio the predictions were dominated by the fleet signal — which is the entire point of doing federated learning in the first place — while still reflecting the individual user.

Iteration 6 — Final per-vehicle-type training results

After all the above, the live system produces the per-vehicle-type accuracy numbers shown on the Fleet Dashboard's *Model Insights* page after 2 FL rounds and 6 device uploads.

Figure 6 — Per-vehicle-type local-model accuracy from the deployed system



Two patterns are worth noting from these real numbers:

1. **Real-data vehicles outperform synthetic ones.** Tesla, BMW, and Hyundai (real CSV) all sit above 46 %; Chevy and Nissan (synthetic) sit around 32 %. This is consistent with the intuition that synthetic data with matched marginals still loses the joint structure of the real dataset.
2. **Loss does not track accuracy linearly.** Chevy has the highest accuracy among the synthetic pair *and* the highest loss; Nissan has the lowest of both. The 8-parameter linear model is shallow enough that loss and accuracy are partially decoupled — the loss surface is dominated by a few high-leverage features (battery temperature, depth of discharge), while accuracy reflects whether the prediction crosses the service-required threshold.

These numbers are themselves the strongest argument that the system is doing real work: a well-implemented FL pipeline operating on real EV data produces a measurable, vehicle-type-dependent learning signal, while preserving the privacy properties laid out in §A.2.

Closing Notes

What was real and what was qualitative

To keep the methodology honest:

- **Real and verifiable from the live system:** dataset sizes (Figure 1), the per-vehicle-type accuracy numbers (Figure 6), the architectural change in Iteration 2, the σ = 0.3 / threshold = 3 / 70 / 30 settings.
- **Qualitative observations from development:** the σ trade-off curve (Figure 3), the threshold trade-off (Figure 4), the blend-ratio curve (Figure 5). These reflect the behaviour we saw while iterating on the system, not a controlled benchmark.

A formal benchmark — sweeping σ, threshold, and α with held-out test sets and reporting (ε, δ) budgets per round — is acknowledged future work and would be the natural next contribution if the project were extended.

Why this design lands where it does

Every setting in the final system corresponds to an axis on the FL design space surveyed in Part A:

Final setting	Design axis	Alternatives rejected
FedAvg	Aggregation algorithm	FedProx, FedMedian, Krum
Gaussian noise σ = 0.3 + L2 clipping	Privacy mechanism	Plain, Laplace, secure aggregation, homomorphic
Cross-device with 5 demo phones	Topology	Cross-silo
5 per-type globals + 70 / 30 personalisation	Heterogeneity	One global, pure personalisation, clustered FedAvg
Threshold = 3	Communication	Threshold = 1, threshold = 5+, partial participation

The resulting system is not the most technically sophisticated FL implementation on any single axis, but it is internally consistent: every choice is the version of that axis that fits a 6-user, 5-vehicle-type, mobile-deployed prototype where every decision had to be inspectable in the report.